

Implementing Predictive Maintenance

For Small and Medium-Sized Enterprises

Ingela Holmkvist

Bachelor Thesis

Main field of study: Computer engineering

Credits: 15 hp

Semester/year: Spring 2024

Supervisor: Qing He

Examiner: Stefan Forsström

Course code: DT099G

At Mid Sweden University, it is possible to publish the thesis in full text in DiVA (see appendix for publishing conditions). The publication is open access, which means that the work will be freely available to read and download online. This increases the dissemination and visibility of the degree project.

Open access is becoming the norm for disseminating scientific information online. Mid Sweden University recommends both researchers and students to publish their work open access.

I/we allow publishing in full text (free available online, open access):

- ☒ Yes, I/we agree to the terms of publication.
- ☐ No, I/we do not accept that my independent work is published in the public interface in DiVA (only archiving in DiVA).

Sundsvall, 2023-05-16

.....
Location and date

Datateknik 180 hp / DT099G
.....
Programme/Course

Åsa Ingela Holmkvist.....
Name (all authors names)

1986-03-15.....
Year of birth (all authors year of birth)

Abstract

Maintenance has always been an important aspect for industrial companies, with new technologies allowing for predictive maintenance to be an option. Collected sensor data, and other data, can in this case be used to improve maintenance to reduce costs and increase competitiveness for the company. Small and medium-sized enterprises (SMEs) are important to the economy with 99% of all businesses in Europe being considered an SME. It's important for small and medium-sized companies to also be included in the implementations of predictive maintenance due to their great number and importance to the economy. The aim of this thesis has been to investigate implementation challenges for SMEs that want to use predictive maintenance. The problem has been to identify challenges and possible options for smaller companies by evaluating and comparing several machine learning algorithms. The problem investigated in this thesis is identifying a machine learning algorithm that optimizes both performance and resource use. By making predictive maintenance more approachable for SMEs, they can also benefit from a reduction in operational costs, extended lifespan of their equipment and remain competitive in the future. Five machine learning models underwent training and testing. The algorithm that performed best was XGBoost, with Random Forest being a close contender. However, if resources are very limited, Decision Trees perform best out of the simpler models.

Keywords: Machine Learning, Predictive Maintenance, SME, Classification.

Sammanfattning

Underhåll har alltid varit viktigt för industriella företag, med nya teknologier som prediktivt underhåll på väg fram. Insamlad sensordata och annan data används för att förbättra underhållet och för att minska kostnader och öka konkurrenskraften för företaget. Små och medelstora företag (SME) är viktiga för ekonomin, med 99 % av alla företag i Europa betraktade som SMEs idag. Det är viktigt att små och medelstora företag också inkluderas i implementeringen av prediktivt underhåll på grund av deras stora antal och betydelse för ekonomin. Syftet med denna avhandling har varit att undersöka utmaningar i implementering för SME som vill använda prediktivt underhåll. Problemet har varit att identifiera utmaningar och möjliga alternativ för mindre företag genom att utvärdera och jämföra flera maskininlärningsalgoritmer. Problemet som undersökts i avhandlingen har varit att identifiera en maskininlärningsalgoritm som optimerar både prestanda och resursanvändning. Genom att göra prediktivt underhåll mer tillgängligt för SME kan de också dra nytta av minskade driftskostnader, förlängd livslängd på sin utrustning och förbli konkurrenskraftiga i framtiden. Fem maskininlärningsmodeller tränades och testades. Algoritmen som presterade bäst var XGBoost, med Random Forest som en nära utmanare. Om resurserna är mycket begränsade presterar Decision Trees bäst av de enklare modellerna.

Nyckelord: Maskininläring, Prediktivt Underhåll, SME, Klassificering.

Acknowledgements

I want to acknowledge my supervisor at Mid Sweden University, Qing He for her great support and guidance throughout the entire project. I always came out of our meetings with new ideas and inspiration.

I would also like to thank everyone at Knowit AB in Sundsvall for opening their doors to me and supporting me during the thesis project. Especially my supervisor Andrea Jacobsson for listening and offering feedback and being available for thoughts throughout the entire project. Additionally, I would also like to thank Olise Östmark at Knowit for handling a lot of the practical aspects and making me feel very welcome.

Contents

Sammanfattning.....	4
Acknowledgements	5
Terminology.....	9
1 Introduction	10
1.1 Background and motivation.....	10
1.2 Overall aim and problem statement	11
1.3 Scientific goals	11
1.4 Scope	11
1.5 Outline	12
2 Theory.....	13
2.1 Maintenance.....	13
2.1.1 Reactive Maintenance	13
2.1.2 Preventive Maintenance	14
2.1.3 Predictive Maintenance	15
2.2 Machine Learning	16
2.2.1 Machine Learning Types	16
2.2.2 Standardization.....	17
2.2.3 Data Splitting	17
2.3 Machine Learning Algorithms.....	18
2.3.1 Logistic Regression.....	18
2.3.2 Decision Trees	19
2.3.3 Random Forest.....	19
2.3.4 Support Vector Machines	20
2.3.5 XGBoost	20
2.4 Performance Evaluation.....	22
2.4.1 Accuracy	22
2.4.2 Precision.....	22
2.4.3 Recall	24
2.4.4 F1 Score	24
2.4.5 ROC-AUC.....	24
2.4.6 Confusion Matrix.....	26
2.5 Related work.....	26
2.5.1 Predictive Maintenance	26
2.5.2 Predictive Maintenance for SMEs	27
2.5.3 Dataset.....	28
3 Methodology	29
3.1 Scientific method description.....	29

3.2	Project method description.....	29
3.2.1	Literature Study.....	30
3.2.2	Evaluation of Algorithms.....	30
3.3	Project evaluation method.....	31
4	Choice of Approach.....	32
4.1	Requirements capturing.....	32
4.1.1	Technical Setup and Development Environment.....	32
4.1.2	Dataset and preparation.....	32
4.1.3	Machine Learning Algorithms	32
4.2	Requirement analysis	33
4.2.1	Technical Setup and Development Environment.....	33
4.2.2	Dataset and preparation.....	33
4.2.3	Machine Learning Algorithms	34
4.3	Chosen Approach	34
4.3.1	Technical Setup and Development Environment.....	34
4.3.2	Dataset and preparation.....	35
4.3.3	Machine Learning Algorithms	36
5	Implementation	37
5.1	Dataset Preparation	37
5.2	Creating and Training ML Models.....	38
5.3	Evaluation	39
5.4	Hyperparameter Optimization.....	40
6	Results	41
6.1	Literature study.....	41
6.2	Imbalanced Dataset	41
6.3	Base algorithm Testing.....	42
6.3.1	ROC-AUC Curves	44
6.3.2	Confusion Matrices	44
6.4	Hyperparameter Optimization.....	48
7	Discussion.....	52
7.1	Analysis and discussion of results	52
7.1.1	Base Algorithm Testing	52
7.1.2	Hyperparameter Optimization.....	53
7.2	Project method discussion.....	53
7.3	Scientific discussion.....	55
7.3.1	Analyzing the Results	55
7.3.2	Synthetic Datasets.....	55
7.4	Consequence analysis.....	56
7.5	Ethical and societal discussion.....	56

8	Conclusions	58
8.1	Future work	58
	References	59
	Appendix A: ROC-AUC Curves.....	1
	Appendix B: Source Code.....	4

Terminology

A couple of the terms in the report have been put together as abbreviations to save space.

Acronyms/Abbreviations

AI	Artificial Intelligence.
DT	Decision Trees.
HDF	Heat Dissipation Failure.
LR	Logistic Regression.
ML	Machine Learning.
OSF	Overstrain Failure.
PdM	Predictive Maintenance.
PWF	Power Failure.
RNF	Random Failure.
RF	Random Forest.
SME	Small and Medium Enterprises.
SVM	Support Vector Machines.
TWF	Tool Wear Failure.
XGB	XGBoost.

1 Introduction

This study has explored the possibilities and challenges of implementing predictive maintenance for small and medium-sized enterprises. The topic of predictive maintenance was suggested by Knowit AB, and the work has been done in cooperation with them at their office in Sundsvall.

1.1 Background and motivation

Maintenance has always been an important aspect for industrial companies, though the way it's done has changed over time. Traditionally production would continue until a component broke and had to be repaired or replaced. Several new technologies are having a big impact on how businesses are run as we move forward, and the potential of them is expected to be huge. To improve competition, companies are increasingly implementing digital technologies, intelligent machines for more flexible production, integration and communication throughout the production and even customer chain [1]. One such technology is predictive maintenance (PDM). Collected sensor data, and other data, can in this case be used to improve maintenance to reduce costs and increase competitiveness for the company [2].

Unplanned maintenance is a huge cost for companies overall, there is both the cost of the maintenance itself but also the sometimes much bigger cost of losing production time. The market of predictive maintenance has been growing quickly, with a growth of 11% from 2021 to 2022 [3]. The growth is expected to continue and is predicted to reach 32.30 billion USD by the year 2030 [4].

Small and medium-sized enterprises (SMEs) are important to the economy with 99% of all businesses in Europe being considered an SME [5]. There are two main areas that determine whether a company should be classified as a SME, the number of employees and either turnover or balance sheet total. A company is considered an SME if it has less than 250 employees and an annual turnover less than 50 million euro and/or annual balance sheet of less than 43 million euro [6].

It's important for small and medium-sized companies to also be included in the implementations of predictive maintenance due to their great number and importance to the economy.

1.2 Overall aim and problem statement

The aim of this thesis has been to investigate implementation challenges for SMEs that want to use predictive maintenance. The problem has been to identify challenges and possible options for smaller companies by evaluating and comparing several machine learning algorithms.

The problem investigated in this thesis has been to identify a machine learning algorithm that optimizes both performance and resource use. By making predictive maintenance more approachable for SMEs, they can also benefit from a reduction in operational costs, extended lifespan of their equipment and remain competitive in the future.

1.3 Scientific goals

The goal of this project is to:

1. Gather information on the following topics:
 - What is Predictive Maintenance?
 - See if there are any existing case studies and applications.
 - Identify potential challenges for small and medium sized companies.
 - Technical aspects, AI techniques and algorithms.
 - Identify tools and platforms.
2. Evaluate which machine learning algorithm offers the best performance when it comes to accuracy and effectiveness in predicting equipment failures.
3. Evaluate how the different algorithms compare in terms of memory use, training time and prediction time in order to see how well they are suited for SMEs.

1.4 Scope

This thesis will focus on predictive maintenance for SMEs, examining what predictive maintenance is and what challenges it might have for smaller companies. The study will mention other forms of maintenance. However, they will only be mentioned in short. While this project

employs AI through machine learning algorithms, its primary focus is specifically on evaluating predictive maintenance algorithms suitable for SMEs.

The study will focus on evaluating specific machine learning algorithms using a public dataset.

Although there are economic interests in implementing predictive maintenance, this thesis will not perform a detailed economic analysis. The project is focused on the technical aspects of machine learning algorithms.

1.5 Outline

Chapter 2 describes the theoretical topics that are needed to understand this thesis. It addresses topics such as different types of maintenance, machine learning, algorithms, and ways to evaluate them. Chapter 3 presents an overview of the methods used during this project, such as a literature study and evaluation of ML algorithms. Chapter 4 presents different choices that had to be made, and what options got chosen in the end. Chapter 5 describes the technical implementation, first with an overview and then a more detailed introduction to each part. Chapter 6 shows the results with graphs for visualization. Chapter 7 includes a discussion about the results as well as a few other topics, and finally chapter 8 concludes the report with thoughts on future work.

2 Theory

This chapter presents theoretical information and concepts related to the topics of this thesis.

2.1 Maintenance

Maintenance is very important for companies in production. Machine breakdowns during production can disrupt schedules, lead to delivery delays, and often require employees to do overtime to make up for the lost time. There are several types of maintenance, each with their own advantages and disadvantages.

2.1.1 Reactive Maintenance

Reactive maintenance, or run-to-failure maintenance, simply runs the equipment until it breaks and performs maintenance or replaces the component at that point. This form of maintenance does not require any planning but can become costly with the unplanned interruptions and delays [7, 8, 9]. A crucial component breaking down in one end of the company might halt production in other areas as well, and if the component needs to be delivered and installed by a professional the cost could be significant for the company. Components that aren't regularly maintained might also perform worse and increase costs. However, if the component is non-critical to the production this type of maintenance might still be a good option since there is less planning required [9]. Figure 1 shows the advantages and disadvantages of reactive maintenance.

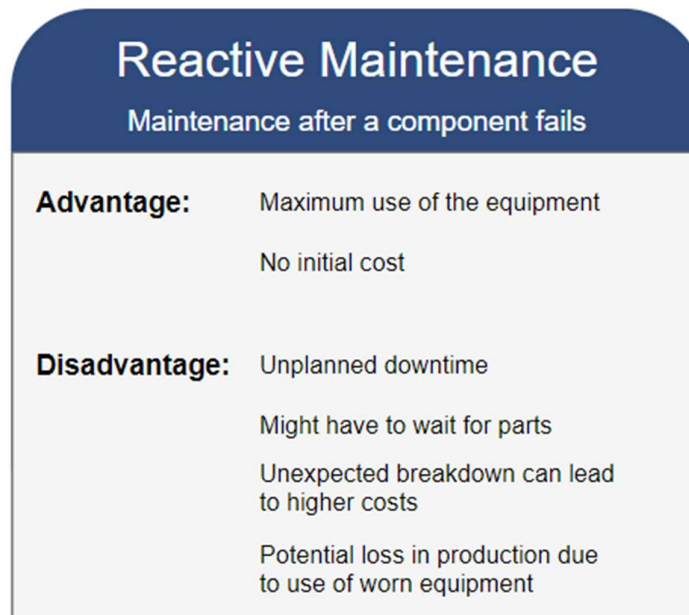


Figure 1. Advantages and disadvantages of reactive maintenance.

2.1.2 Preventive Maintenance

With preventive maintenance the maintenance is performed regularly based on previous experience and observations. This approach involves scheduled inspections, service, and repairs to detect and correct any issues before they become serious problems or break down. The goal is to minimize downtime, reduce repair costs, improve safety, and make sure everything runs effectively [7, 9].

There are some drawbacks to preventive maintenance as well. The method is excellent for planning maintenance and avoiding long and costly downtimes to production, but components are often maintained before there is an actual need to do so. This takes time and money and might be an unnecessary cost for the company [7]. Figure 2 shows the advantages and disadvantages of preventive maintenance.

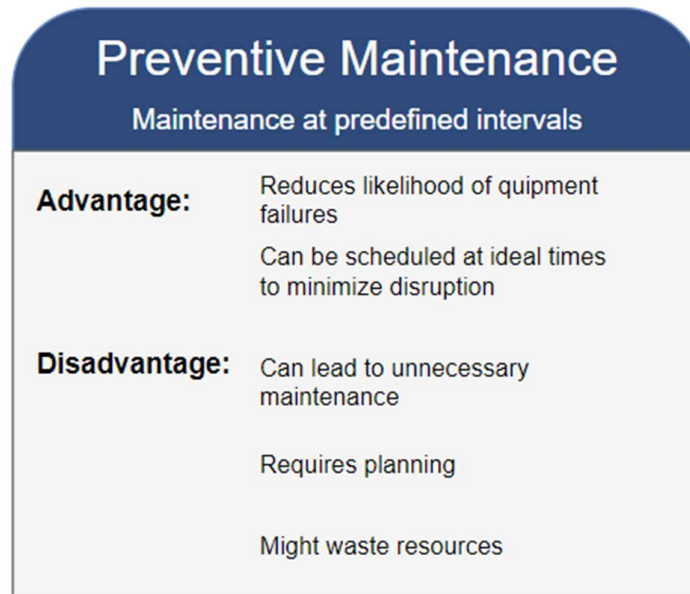


Figure 2. Advantages and disadvantages of preventive maintenance.

2.1.3 Predictive Maintenance

The objective of predictive maintenance is to optimize the timing of maintenance so that it's done as necessary. Ideally maintenance is neither too late, as often happens with reactive maintenance. And not too early, which could be the case with preventive maintenance [7]. It's not only possible to monitor for optimal maintenance times, but also to detect early signs of unusual wear or other problems [10].

Predictive maintenance is often implemented with systems like the Internet of Things (IoT), where sensors continuously monitor and analyze data to check the status of the equipment. The sensors gather data on relevant parameters, which could be things such as temperature, pressure, rotation speed or sound. These sensors can automatically detect and alert when things deviate from what's normal and make it possible to take immediate actions [10, 11].

High-quality data is crucial for PdM modeling. After gathering the raw data from the sensors, it will usually have to be preprocessed in order to handle noise or inconsistencies. It involves data cleaning, standardization, and the management of missing data. AI algorithms are used to analyze data to predict failures based on historical data. Decision making modules can then use the insights and predictions from the AI

algorithms to recommend when maintenance should be performed [11]. Figure 3 shows the advantages and disadvantages of predictive maintenance.

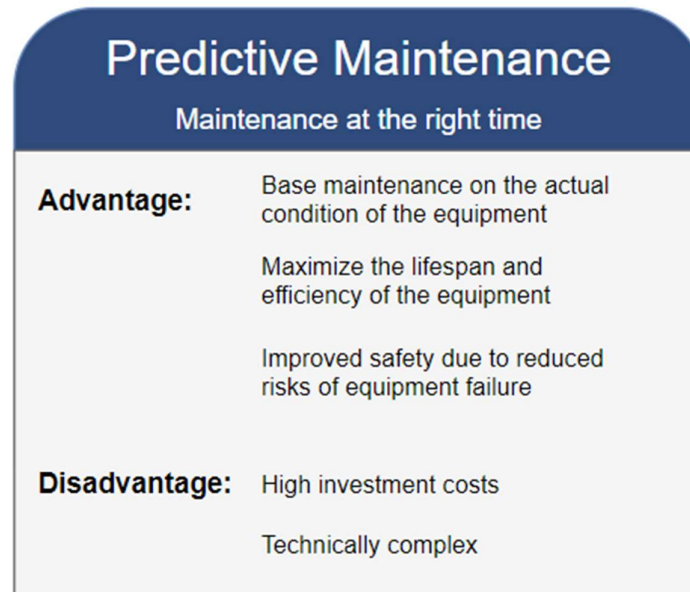


Figure 3. Advantages and disadvantages of predictive maintenance.

2.2 Machine Learning

Machine Learning (ML) is a great tool for making predictions of future events based on collected historical data. After the data has been collected into datasets, the computer uses statistical methods to analyze data and identify patterns. The computer makes predictions from the data, assessing the accuracy against known outcomes. The predicted results are constantly refined by adjusting if there is an error. This ability to self-improve allows the algorithm to discover insights and improve its accuracy over time, without human assistance [12].

2.2.1 Machine Learning Types

There are several types of machine learning, two of the main types are supervised learning and unsupervised learning.

Supervised learning trains the algorithm with labeled data, where the correct output is known. The model learns by making predictions and adjusting based on the differences between its predictions and the actual

outcomes. This method is commonly used for tasks like classification and regression, where the goal is to predict the correct label or value.

Unsupervised learning on the other hand trains a model using data that hasn't been labeled. The algorithm tries to identify patterns in the data on its own. It's useful for clustering and association tasks, where the goal is to group similar data points together or discover rules that describe large portions of the data [13, 14].

Supervised learning is further divided into classification- and regression problems. Classification can be used for discrete variables. The goal is to pair each input with the correct label. It deals with situations where there are only two possible classes for the inputs, while multi-class classification handles a greater number of classes. Regression learning handles outputs with continuous values [15].

2.2.2 Standardization

Before training a ML model it is necessary to standardize the data for some machine learning models. While it's usually not required for Decision Trees, Random Forest and XGBoost, it has some important benefits for Support Vector machines (SVM). Standardization improves SVMs performance since the algorithm relies on calculating the distance between data points, and large features might affect the distances in an uneven way compared to smaller features. Scaling the features to a mean of zero and a standard deviation of one ensures that each feature contributes more evenly to the prediction [16].

2.2.3 Data Splitting

When evaluating the performance of an ML model, it's important that the data used for testing the model is different and unseen compared to the data used for training. The model should be evaluated based on input that it hasn't seen before. Splitting the dataset might also help with avoiding overfitting. Overfitting can happen if the predictions of a model are too specialized on the training data. Instead of making predictions based on relevant patterns, it could also make very specialized predictions on isolated situations (perhaps events that lead to a machine failure, but only once). This can lead to a model that is very good at predicting results of the training data but performs poorly on new data. A common way to handle this is by splitting the dataset into two parts,

one used exclusively for training and the other one for testing [17], as seen on figure 4.

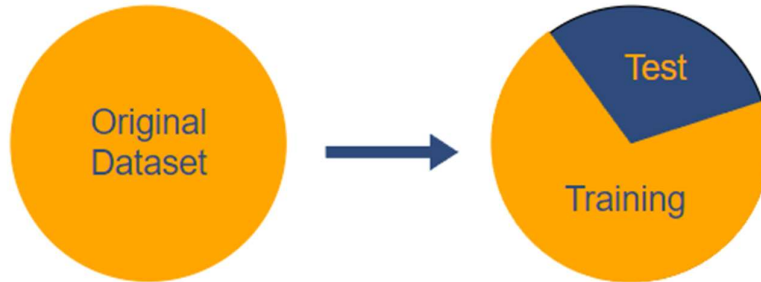


Figure 4. The dataset is split into separate training and testing data.

2.3 Machine Learning Algorithms

This chapter presents the different machine learning algorithms used in the project.

2.3.1 Logistic Regression

The logistic regression in Scikit-learn models the probabilities describing the possible outcomes of a single trial using a logistic function [18]. It's a supervised machine learning algorithm that is often used for binary classification tasks. The algorithm works by modeling the probability that a given input point belongs to a certain class. The result will be the likelihood of the input belonging to a class with a probability between 0 and 1 [19]. For the formula for logistic regression, see equation 1 [20]:

$$y = \frac{e^{(b_0 + b_1 X)}}{1 + e^{(b_0 + b_1 X)}} \quad (1)$$

- x = input value
- y = predicted output
- b_0 = bias or intercept term
- b_1 = coefficient for input (x)

This method repeatedly tests different values for the coefficients to find the best match that accurately predicts the data. Once the model is ready, it can predict the chance of a result. These chances are then used

to sort observations by setting a cutoff point, 0.5 is a common cutoff point. For example, if the chance is greater than 0.5, the model predicts 1. If it's less than 0.5, it predicts 0 [19].

2.3.2 Decision Trees

A decision tree is a supervised learning algorithm used in both classification and regression tasks. It has a hierarchical tree structure, including a root node, branches, internal nodes, and leaf nodes. The root node begins the tree and branches out to internal nodes that sort the data based on conditions. The final result is represented by the leaf nodes which present all the possible outcomes.

Decision trees use a divide-and-conquer method, performing a greedy search to find optimal split points. The data is sorted by running it from the root node, making decisions at each split-point until a leaf node has been reached. This is repeated until all data points have been sorted into specific classes. The goal is to create small, simple trees as larger trees can lead to overfitting. This problem might be addressed by pruning unnecessary branches to simplify the model. Decision trees are flexible and rather easy to understand, though they can suffer from issues like overfitting and high variance [21].

2.3.3 Random Forest

The random forest algorithm can handle both classification and regression tasks. It's a form of ensemble learning, where multiple decision trees are combined to produce a more accurate and stable prediction. Instead of relying on the result from just one tree, the results from multiple ones are combined. Each tree in the collection is built from a random sample of the data, adding to the diversity in the models.

Each tree uses feature randomness so that they will have a different subset of features. This reduces the similarities between the trees and improves the overall accuracy of the model. This method is known as feature bagging and helps with mitigating overfitting by making individual predictions that only provide a part of the total prediction. So, if one of the trees makes poor choices by picking the wrong features to be compared, this will have a smaller effect on the total result compared to a decision tree where it would have all the say in the prediction. The final prediction in a random forest is made by averaging the outputs (for

regression problems) or taking a majority vote (for classification problems) across all trees.

Despite its effectiveness and flexibility, random forests can be complex and more costly in the form of resources. Particularly when dealing with very large datasets [22].

2.3.4 Support Vector Machines

Support Vector Machines (SVMs) are a type of supervised machine learning algorithm for classification tasks. The algorithm works by finding an optimal line which acts as a decision boundary between different classes. This is done by maximizing the margin between the nearest data points of each class.

SVMs can manage both simple and more complicated classification tasks. They do this through something called kernel functions, like polynomial, radial basis function and sigmoid kernels. These kernels help by turning the data into a form where it can be separated by a simple line. SVMs can be two dimensional or multidimensional depending on the number of features in the input data. The number of dimensions matches the number of features [23].

2.3.5 XGBoost

The XGBoost is a gradient boosting algorithm that can handle both regression and classification problems. It builds a series of simple models, typically decision trees, one after the other. Each new model tries to fix the errors made by the previous ones. This sequence helps in forming a strong overall model from many weak ones. XGBoost is unique because it uses gradient boosting techniques to minimize errors when adding these new models. This means it gradually finds the best way to improve the model with each step.

To avoid overfitting, XGBoost uses a technique called regularization. It also prunes the trees early in the process and stops them from growing after they don't improve anymore. XGBoost is also good at dealing with missing data by assigning default directions to missing values during splits and optimizing imputation values to minimize training loss. The system is designed to be very fast, making use of advanced hardware optimizations and parallel processing to train models quickly [24].

2.4 Performance Evaluation

The performance of machine learning algorithms can be evaluated using several different evaluation and visualization methods. The following components are used to calculate the performance metrics.

- True Negative (TN): Number of correctly classified negative cases.
- True Positive (TP): Number of correctly classified positive cases.
- False Negative (FN): Number of incorrectly classified negative cases.
- False Positive (FP): Number of incorrectly classified positive cases.

2.4.1 Accuracy

Accuracy measures the ratio of correctly predicted cases compared to the total number of test cases. It provides information about how often the model's predictions are correct. While accuracy is a commonly used metric, it can be inaccurate for imbalance datasets due to one class having significantly more samples [25, 26]. The formula for calculating accuracy can be seen in equation 2.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

2.4.2 Precision

Precision can be used to measure the accuracy of the positive predictions. This shows how many of the cases that when predicted to be positive, were actually positive. A high precision means that there are few false positive predictions, so that when there are positive outcomes, they are likely to be correct. A low precision on the other hand could mean that the model is incorrectly labeling negative cases as positive. This leads to a higher rate of false positives [26, 27]. Equation 3 shows how precision is calculated.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

2.4.3 Recall

Recall measures the ability to correctly identify positive predictions from all test cases. It checks how many of the true positives were actually predicted as true. A high recall indicates that the model is effectively identifying most of the positive instances in the dataset. While a low recall suggests that the model is missing a lot of positive cases [26, 27]. Figure 4 shows the equation for recall.

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

2.4.4 F1 Score

The F1 score is especially useful for evaluating the performance of imbalance classes, or when both prediction and recall are important. It calculates the harmonic means of precision and recall, providing a single value that balances both metrics. This offers a balance between the two metrics, ensuring that both false positives and false negatives are considered. A high F1 score indicates both high precision and high recall, meaning that it is making accurate positive predictions while also capturing most of the true positives in the dataset. A low F1 score suggests that the model is either missing many positive instances or making many false positive predictions [26, 27]. The formula for calculating F1 Score can be seen in equation 5 below.

$$F1\ Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5)$$

2.4.5 ROC-AUC

This metric evaluates the performance based on how well positive and negative cases are separated. It measures the algorithm's ability to correctly identify positives while minimizing false positives. A high ROC-AUC suggests good separation of positive and negative cases, which means better performance. The ROC-AUC score is displayed as an area under the curve. The range is between 0 and 1, with a higher score being better and 1 representing a perfect fit [26]. The chart is drawn by

plotting the calculated result of the true positive rate and the false positive rate for each threshold [28].

2.4.6 Confusion Matrix

A confusion matrix is a table that displays the number of true positives, true negatives, false positives, and false negatives [26]. As seen in figure 5, the rows indicate the true label (if a case is actually true or not), and the columns represent the predicted result. The values on the diagonal, from top-left to bottom-right, represent the cases predicted correctly. Each diagonal element is the sum of all true predictions, both positive and negative.

True Label	No Failure	Failure
	No Failure	Failure
No Failure	True No Failure	False Failure
Failure	False No Failure	True Failure

Figure 5. Confusion matrix with each of its areas.

2.5 Related work

Several previous works have been useful when researching and working on this project.

2.5.1 Predictive Maintenance

The paper Predictive Maintenance and Intelligent Sensors in Smart Factory: Review focuses on how new technologies are changing the way maintenance is handled. A combination of IoT, machine learning and data analysis can be a new way of improving maintenance. They look into how predictive maintenance can improve factories and make them

smarter and more reliable. The paper gives a good idea of what predictive maintenance is in the bigger picture (10).

A review by Thyago P. Carvalho and colleagues investigates the application of machine learning for predictive maintenance. They study presents the different categories of maintenance, reactive maintenance, preventive maintenance, and predictive maintenance. The paper also points out the importance of data quality and algorithm selection. Overall, this paper was very useful for understanding the different types of maintenance and some challenges when applying ML to predictive maintenance [7].

2.5.2 Predictive Maintenance for SMEs

Several related works have been looked at to gain insight into implementing predictive maintenance systems for SMEs.

A study on SMEs in the UK indicates that predictive maintenance is the most economic option, however the initial investment cost is significant. The study also points out that due to the significant investment cost it will be beneficial for smaller companies to collaborate and see if some resources can be shared [29].

A study by Kwaku Adu-Amankwa and colleagues focuses on developing a predictive maintenance cost model for SMEs operating CNC machine tools. The researchers collected data through surveys from SMEs, modeling these datasets to propose a cost-effective PdM system. The study finds that while predictive maintenance is the most cost-effective maintenance strategy, it involves high initial investment costs. The high initial costs include investments in sensors, data analytics tools, and training staff. These expenses can be substantial, particularly for smaller companies with limited resources. The study suggests that SMEs could benefit from pooling resources to optimize their maintenance costs [30].

Another study by the International Journal of Advanced Manufacturing Technology also indicates that initial investment costs might be challenging for SMEs. Research also shows that there are several other significant challenges for SMEs that want to implement predictive maintenance systems, such as the need for technical expertise. Companies might lack the expertise to manage and analyze the data, a

problem that might be more likely to occur in small companies that tend to be very specialized in certain areas. Handling the large volumes of data generated by PdM systems demands robust data management and storage solutions, this is another area of expense that needs consideration [31].

According to a mapping study by M. Khan et al there is a limited number of studies focused on SMEs. The study shows that there is a need for new methods for implementing PdM. Another problem is the limited number of public datasets that are available and can be used for training machine learning models. For future works they point out that available data can be used to make PdM more optimized for SMEs [32].

2.5.3 Dataset

One way to handle the limited access to public datasets is the creation and sharing of public synthetic datasets. For many companies it can be difficult to publish datasets based on the company's real data, so synthetic datasets offer a dataset that reflects realistic data as well as possible. The synthetic dataset AI4I 2020 Predictive Maintenance Dataset has been used in this project and is presented in the paper from Stephan Matzka [33].

3 Methodology

This chapter aims to describe the methodology used during the project to compare various machine learning algorithms for predictive maintenance, focusing on SMEs.

3.1 Scientific method description

This study is grounded in quantitative research methodology, which is common in the field of computer engineering. It relies on gathering and analyzing numerical data [34]. This approach is particularly suitable for this thesis, as the primary aim is to analyze the performance of various machine learning algorithms in terms of accuracy and computational efficiency, which are quantifiable metrics.

To achieve the first goal of the project, to gather more information, a literature review was performed. The information gathered during this part of the project is crucial when selecting the approach for which algorithms to test and how to perform the evaluation.

The second goal is to perform evaluations on the selected ML algorithms. This is done by running each algorithm five times and presenting the average result. Each algorithm will be evaluated based on accuracy, precision, recall, F1 score, confusion matrix and ROC-AUC curve.

The third goal includes comparing memory use and other aspects that might be interesting for SMEs. This is done by measuring the training and testing times for each algorithm and measuring the memory use during training of the ML model.

3.2 Project method description

The goals of this thesis are to conduct a thorough literature review on predictive maintenance and to empirically evaluate several machine learning algorithms using a dataset relevant to SMEs. This approach allows for a comprehensive understanding of the field as well as a practical examination of potential solutions to the challenges identified through the literature.

3.2.1 Literature Study

During the literature study relevant material was collected and read through to gain knowledge about machine learning, predictive maintenance, and SMEs. The sources are research papers, e-books, articles, and other materials. The sources were found using the search service at the Mid Sweden University library and Google.

The literature study helped in understanding the topic and challenges faced by smaller companies interested in implementing predictive maintenance. It also provided methods for selecting suitable algorithms, information about how the algorithms work and ways to measure their performance. Recent publications have been prioritized over old ones in the literature review.

3.2.2 Evaluation of Algorithms

Following the literature study, there will be an evaluation of suitable machine learning algorithms for SMEs. This involves technical setup, dataset preparation, algorithm selection, performance evaluation and computational efficiency analysis.

- **Technical setup and Development Environment:** The selected development environment is visual studio code, using the programming language Python and its libraries Pandas, Scikit-learn, Psutil, Time, Matplotlib and Seaborn.
- **Dataset Preparation:** The dataset ai4i will be used for this project. The preparations consist of checking the dataset for missing values, testing if it's imbalanced and splitting it into training and testing data.
- **Algorithm Selection:** The machine learning algorithms selected for evaluation are Logistic Regression, Decision Trees, Random Forest, Support Vector Machines and XGBoost.
- **Performance Evaluation:** The algorithms are implemented and evaluated by their performance. They are compared in regard to accuracy, precision, recall, F1 score, confusion matrix and ROC-AUC curve.

- **Efficiency Analysis:** The different algorithms are tested in order to compare them in terms of memory use and how long it takes to train and use them for prediction.

3.3 Project evaluation method

The project will be considered successful based on if any algorithms can be considered useful to solve possible challenges for SMEs. The usefulness of the algorithms will be based both on performance when predicting if a machine failure has occurred or not, but also on how long it takes to train the model and make predictions. Memory use of the models during training will be considered as well.

4 Choice of Approach

This section of the project describes the approach selected to address the scientific goals of the project. It looks at the various alternatives and the approach that was chosen.

4.1 Requirements capturing

This chapter discusses the requirements for the technical setup, development environment, dataset and machine learning algorithms.

4.1.1 Technical Setup and Development Environment

When selecting the programming language for the project it was important that it is a language commonly used for ML modeling and that it has a good selection of useful libraries, both for testing the algorithms and visualizing the results. Python is a very popular language for machine learning due to its simplicity and good selection of libraries and frameworks. Another option that was considered is the language R, which is especially popular among statisticians and has good tools for visualization.

4.1.2 Dataset and preparation

The requirements for the dataset were that it is a complete dataset linked to predictive maintenance. Given the limited timeframe, it was determined in collaboration with the company that collecting and preparing a new dataset from scratch would be too time-consuming. Using a prepared dataset was considered a better option.

4.1.3 Machine Learning Algorithms

When selecting which machine learning algorithms to compare for this study, three main aspects were considered.

The first one being to determine what type of problem it is to predict if a machine failure will occur. Since the goal is to predict whether a machine failure has occurred or not, the result can either be 0 for no failure or 1 for machine failure. This means that predicting if a machine failure will occur is a binary classification problem. The second aspect considered when choosing algorithms is the balance of the dataset. The dataset is considered imbalanced so algorithms that are good at handling imbalanced datasets will be of interest. Finally, according to the literature study there are several potential challenges for SMEs that want to

implement predictive maintenance. Algorithms with effective resource use might be interesting in this regard.

4.2 Requirement analysis

This chapter discusses the different options that was considered for the project.

4.2.1 Technical Setup and Development Environment

Both Python and R are good options, however Python was selected due to prior experience with the language and limited time to learn a new one. Python is also the most popular language, so it's well-suited for beginners and for a SME it might be more likely that they have experience with Python rather than R.

4.2.2 Dataset and preparation

The AI4I 2020 Predictive Maintenance Dataset is publicly available from UC Irvine [35]. The dataset is a complete dataset well related to predictive maintenance. It's created synthetically to reflect real predictive maintenance data as closely as possible based on knowledge and experience. The dataset contains 10,000 data points, each represented as a row with six features arranged in columns. The data points are stored as rows, with columns representing 14 features in columns as seen on figure 6.

Unique identifier (UID) is used for identifying rows and ranges from 1-10,000. Product ID indicates the quality of the product. L for low, M for medium and H for high. Machine failure is a binary target that indicates whether there has been a failure (1) or not (0). There are five different failure modes that the machine failure can consist of.

The five independent failure modes:

- Tool Wear Failure (TWF)
- Heat Dissipation Failure (HDF)
- Power Failure (PWF)
- Overstrain Failure (OSF)
- Random Failure (RNF)

The machine failure label is set to one (failure has occurred), if one or more of the failure modes is true.



Variable	Role	Unit
UID	ID	
Product ID	ID	
Type	Feature	
Air Temp	Feature	K
Process Temp	Feature	K
Rotational Speed	Feature	rpm
Torque	Feature	Nm
Tool Wear	Feature	min
Machine Failure	Target	
TWF	Target	
HDF	Target	
PWF	Target	
OSF	Target	min
RNF	Target	min

Figure 6. The features of the AI4I dataset.

A study related to the dataset indicated that it was an imbalance dataset. [33].

4.2.3 Machine Learning Algorithms

Decision trees, random forest and support vector machines are considered good algorithms for classification problems [36]. For binary and categorical data logistic regression, decision trees and random forest can work well. Random forest, boosted trees and support vector machines can be good when the dataset is imbalanced. For situations where speed and limited resources could be a concern, decision trees might be a good option [37].

4.3 Chosen Approach

This chapter presents the approach that was chosen for the technical setup, development environment, dataset and machine learning algorithms.

4.3.1 Technical Setup and Development Environment

The code was written and debugged in Visual Studio Code, using the programming language Python with its libraries. The Pandas library was used due to its good functionality which significantly simplifies the process of data cleaning, transformation, and preprocessing. The algorithms were implemented with the aid of scikit-learn, a library specifically designed for machine learning in Python. Scikit-learn offers tools for data mining and data analysis. The libraries Matplotlib and Seaborn aid in visualization. The library Time was used for measuring training and prediction time, and Psutil for measuring memory use.

The laptop used for the implementation of the ML models and tests had the following specifications:

- 8 GB RAM
- AMD64 Family 23 Model 104 Stepping 1 AuthenticAMD processor
- Microsoft Windows 11 Home operative system

4.3.2 Dataset and preparation

The AI4I 2020 Predictive Maintenance Dataset was selected for this project. A practical test was done to confirm that the dataset is imbalanced. The dataset was also checked for any null values since missing data can lead to errors during the training process. The dataset was also split into training and testing data before evaluation.

The features selected:

- Air temperature [K]
- Process temperature [K]
- Rotational Speed [rpm]
- Torque [Nm]
- Tool wear [min]

The feature named type was not selected for this project due to it having a very low impact on the predictions in other studies [33]. There was also a limited amount of information about what the type feature was measuring, so its importance to the predictions was considered uncertain.

4.3.3 Machine Learning Algorithms

Algorithms selected for evaluation:

- Logistic Regression (LR)
- Decision Trees (DT)
- Random Forest (RF)
- Support Vector Machines (SVM)
- XGBoost (XGB)

5 Implementation

This chapter describes the technical implementation of the project. Figure 7 provides an overview of the steps taken to build and evaluate the ML models. The chapters following presents each step in further detail.

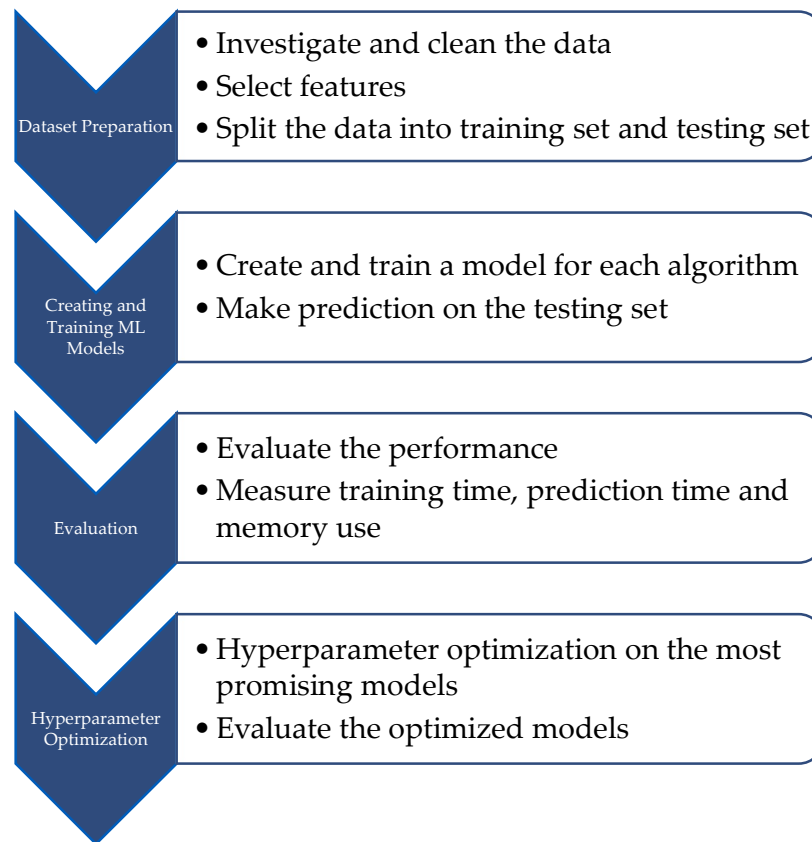


Figure 7. Overview of the project.

5.1 Dataset Preparation

To ensure that the dataset is optimized for the algorithms and to provide a consistent basis for testing, the dataset was prepared in several steps. The dataset was loaded into a panda's data frame, with the same preprocessing steps being taken for each of the algorithms before training them. First a check for missing data since it can lead to errors during the training process. According to UC Irvine there should be no missing values in the dataset [35]. Visual studio code was also used to

check for any null values to verify that there were no missing values in the dataset.

The dataset was tested to see how imbalanced it is, and how the machine learning failure types were distributed within the set. The dataset was scaled with StandardScaler so that each parameter has a mean of zero and a standard deviation of one. This is crucial for many machine learning algorithms. The dataset was split into 80% training and 20% test data, since using the same data for both might lead to overfitting. Splitting the data ensures that the model is tested on unseen data, providing a more accurate evaluation of its predictive performance. Image 8 shows pseudocode for the preprocessing file of the data.

```
Functions:

Load Data(filepath):
    Read CSV from filepath into pandas DataFrame
    Return DataFrame

Preprocess Features(X_train, X_test):
    Initialize a StandardScaler
    Scale X_train and fit the scaler
    Transform X_test using the fitted scaler
    Return scaled X_train, scaled X_test

Get Train Test Data(data):
    Extract target variable 'Machine failure' from data
    Extract feature variables relevant to the analysis
    Split data into training and test sets
    Scale feature variables using Preprocess Features function
    Return scaled training and test feature sets, training and test target sets
```

Figure 8. Pseudocode for preprocessing the data

5.2 Creating and Training ML Models

Each of the models Logistic Regression, Decision Trees, Random Forest, Support Vector Machines and XGBoost were first set up with dataset preparation and then fitted on the portion of data set aside for training. The training time for each algorithm was measured by capturing a timestamp before training started and another after the training was concluded. After training the model a prediction was made on the test data set aside specifically for this. The test data is previously unseen by the model and used to check how well it can predict new data. Prediction

time was measured in the same manner as the training time for each algorithm. The models imported for testing the algorithms were LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, SVC and xgb. Image 9 show pseudocode for the models, a link to the source code can be found in Appendix B.

```
Import: Preprocessing module, Evaluation module

Functions:
    Print Process Usage():
        Get current process memory usage
        Print memory usage in GB

Main Execution Flow:
    Load and preprocess the data using functions from Preprocessing module
    Initialize Decision Tree Classifier with specified hyperparameters
    Print current memory usage
    Time the training process:
        Fit the model on the training data
        Print the training duration
    Print memory usage after training
    Time the prediction process:
        Make predictions on the test data
        Print the prediction duration
    Calculate probabilities of predictions
    Evaluate model using the Evaluation module:
        Calculate and print accuracy, precision, recall, F1 score, ROC-AUC
        Plot ROC curve and confusion matrix
```

Image 9. Pseudocode for the models

5.3 Evaluation

In addition to measuring training and prediction times during the training aspect of the test, other metrics were tested for each model. Accuracy, precision, recall, F1 Score, ROC curve and Confusion Matrix were calculated for each. The libraries used for the metrics were accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve and confusion_matrix. The results for each model were collected five times and an average was calculated for each of them. Matplotlib was used to plot the ROC curves and Seaborn to visualize

each confusion matrix. Training time, prediction time and memory usage were also measured when testing the algorithms.

5.4 Hyperparameter Optimization

After the initial test of the algorithms the most interesting ones will be put through hyperparameter optimization to see if they can be improved further. GridSearch from Sklearn was used to iterate through different hyperparameter settings to find the combination that performs best. The optimized models were measured with the same metrics as the initial test.

The hyperparameters used were the following:

- `n_estimators` - the number of trees
- `max_depth` - to prevent overfitting by limiting how deep the tree can grow.
- `max_features` - the number of features considered for each split.
- `min_sample_leaf` - the minimum number of samples each leaf must have
- `min_samples_split` - the minimum number of samples needed to split an internal node.
- `learning_rate` - shrinks the feature weights after each boosting step
- `subsample` - size of the training dataset to be used for creating each tree.
- `colsample_bytree` - decides the proportion of features used when building each tree.

6 Results

This chapter presents the results from the literature study and test results on the dataset.

6.1 Literature study

The results regarding identifying challenges specific to SMEs are presented here, a more detailed description can be found in chapter 2.5 related works. The initial literature study of the project revealed several challenges for SMEs that want to implement predictive maintenance. Studies suggest that predictive maintenance is likely to be a good choice economically [29]. However, there is a high investment cost [29, 30, 31]. There are indications that it could be beneficial for SMEs to collaborate and share resources when possible [29, 30]. The literature study also revealed that there are several other significant challenges for SMEs. They might have more limitations when it comes to expertise and resources to gather, analyze, and handle the large volumes of data [31]. Availability of data is a common concern, with a limited number of public datasets that can be used to make PdM more optimized for SMEs (32).

6.2 Imbalanced Dataset

According to the check performed the AI4I 2020 dataset is imbalanced with only 339 out of 10,000 data points being labeled as machine failures, as seen on figure 10.

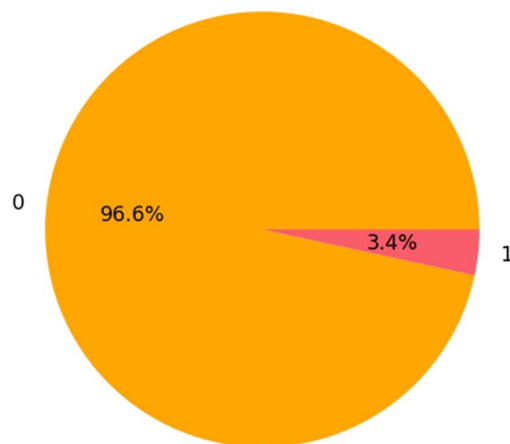


Figure 10. Machine failures only represent 3,4% of the total dataset.

The failures are divided across the machine failure labels as seen in figure 11.

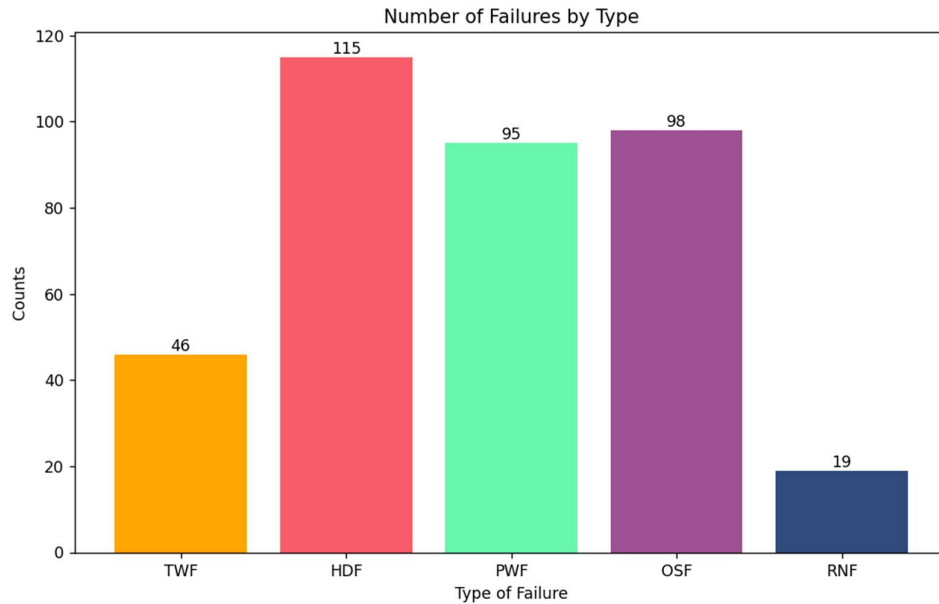


Figure 11. The number of failures of each type.

6.3 Base algorithm Testing

The resulting metrics from the evaluation can be seen in figure 12. All five algorithms had good accuracy during the tests, with Random Forest and XGBoost having a slightly higher percentage. The differences in regard to precision and recall were significant. With precision measuring how well the models identify positive cases. While recall shows how good they are at finding positive cases in the dataset, making sure that no positive cases are missed. RF and SVM measured high precision, while DT got the lowest score. DT, RF and XGB had high recall, while LR and SVM got a very low score on recall. F1 Score balances precision and recall. The best F1 score was shown by RF and XGB. LR on the other hand had a very low F1 Score.

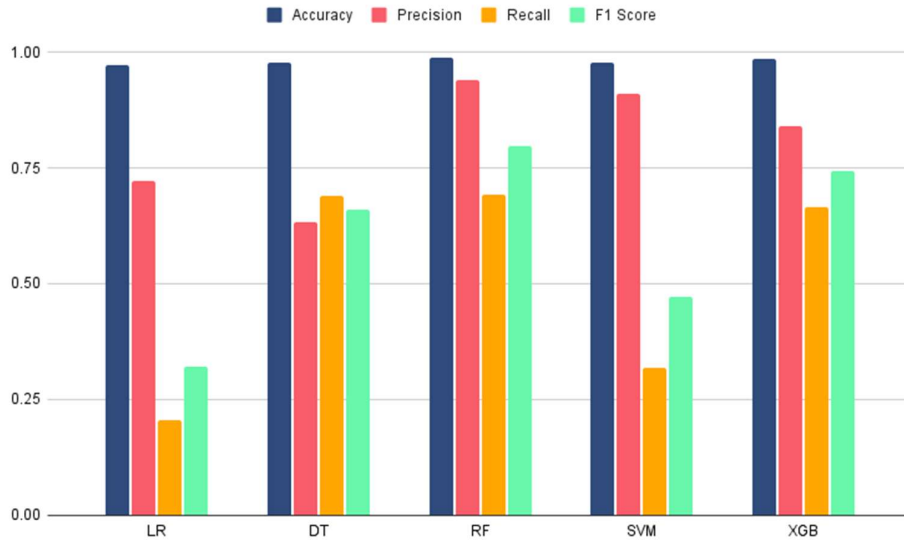


Figure 12. The results of the algorithm tests.

Training and prediction times for the algorithms were measured and are shown in table 13. Logistic Regression and Decision Trees measured very short training and prediction times. Support Vector Machines stand out as slower both when it comes to training of the model and prediction. Random Forest had a relatively slow training time, the prediction time is a bit better.

Model	Training Time(s)	Prediction Time(s)
LR	0.0169	0.000638
DT	0.0182	0.00044
RF	0.653	0.0113
SVM	1.0	0.1
XGB	0.0949	0.00205

Figure 13. The training and prediction times for each algorithm in seconds.

Figure 14 shows the measured memory use of each model during training. XGBoost and Random Forest had a higher memory use compared to the others.

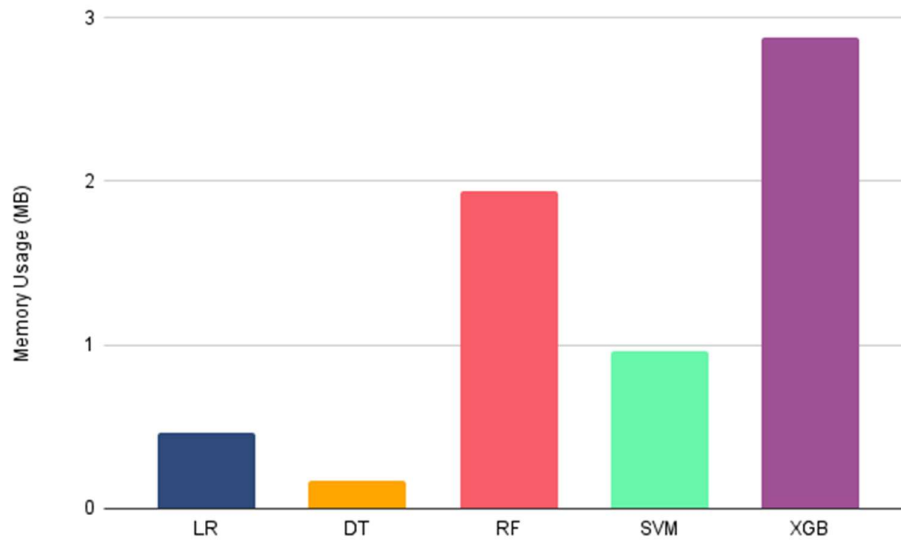


Figure 14. Memory use of each algorithm during training.

6.3.1 ROC-AUC Curves

The results of the ROC-AUC curves are shown in appendix 1. The best performance was XGBoost with AUC of 0.98. Random Forest and Support Vector Machines got an AUC of 0.96, Logistic Regression of 0.93 and Decision Tree the lowest of 0.84.

6.3.2 Confusion Matrices

This section presents the resulting confusion matrices from the evaluation for each algorithm.

Figure 15 shows the confusion matrix for Logistic Regression, where 97.3% of the predictions were correct. There are 55 predictions that were labelled incorrectly, 5 that the model incorrectly marked as “failure” and 50 that was incorrectly marked as “no failure”.

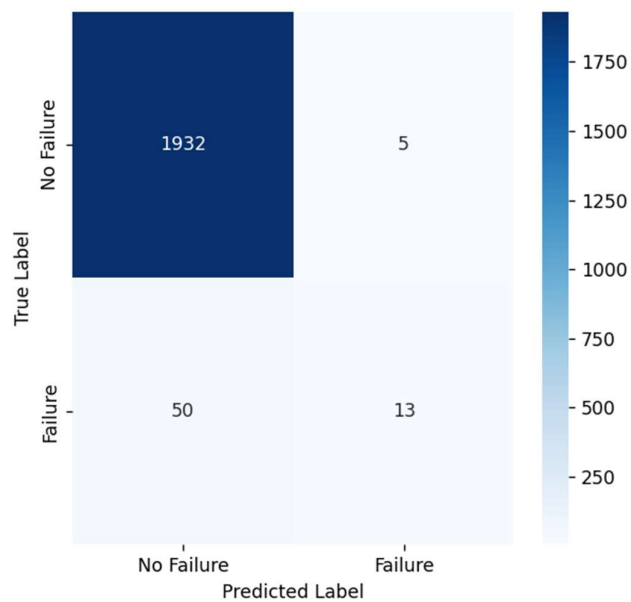


Figure 15. Confusion Matrix for Logistic Regression.

For Decision Trees, as shown in figure 16, the accuracy of correct predictions is 97.8%. There are 24 cases incorrectly labeled as "failure" and 20 cases incorrectly labeled as "no failure."

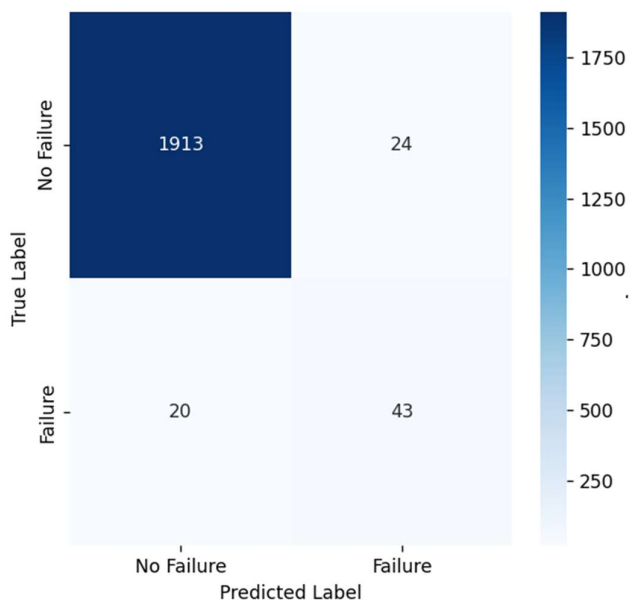


Figure 16. Confusion Matrix for Decision Trees.

For Random Forests, see figure 17, the model accurately predicted 98.9% of the cases. It misclassified 19 cases as "no failure" and 3 cases as "failure."

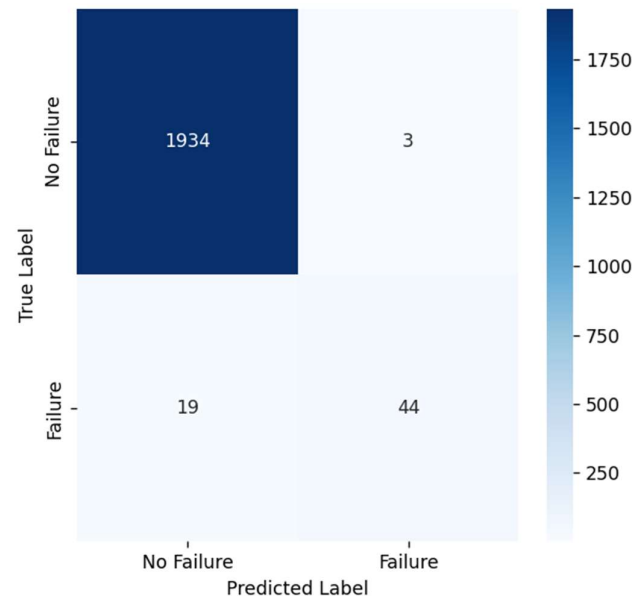


Figure 17. Confusion Matrix for Random Forest.

As shown in figure 18, Support Vector Machines measured an accuracy of 97.8%. The model misclassified 43 cases as "no failure" and 2 cases as "failure."

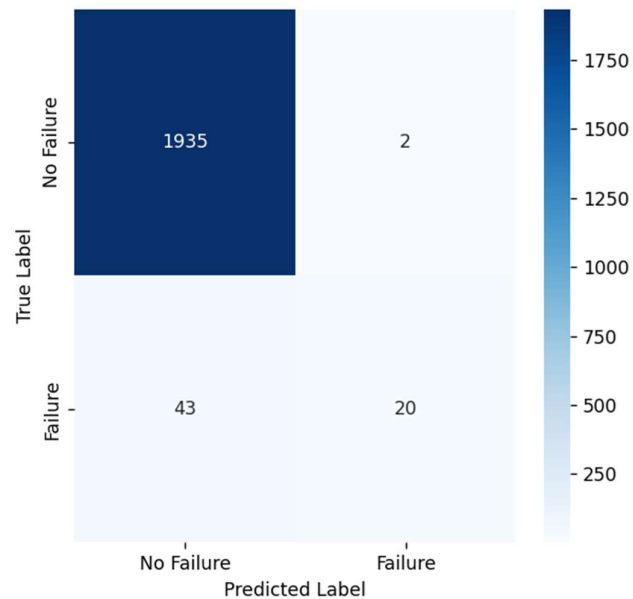


Figure 18. Confusion Matrix for Support Vector Machines.

For XGBoost, as displayed in figure 19, the model correctly predicted 98.6% of the outcomes. There were 21 cases mislabeled as "no failure" and 8 cases incorrectly labeled as "failure."

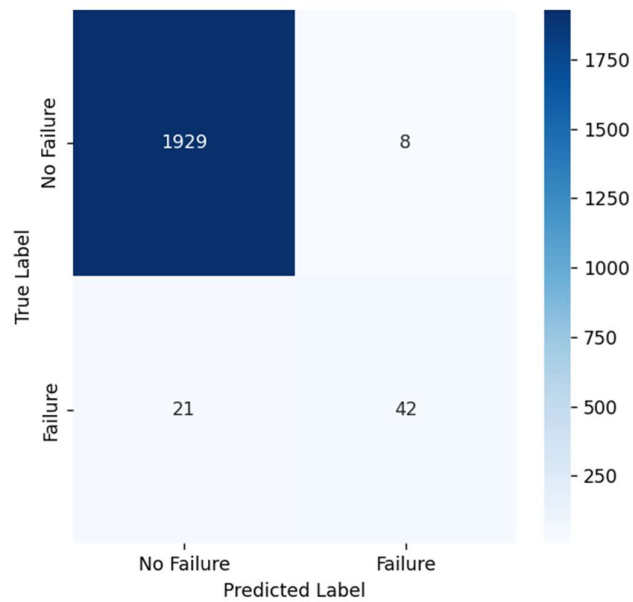


Figure 19. Confusion Matrix for XGBoost.

Figure 20 shows the percentage of “False No Failure” to “False Failure” for each model.

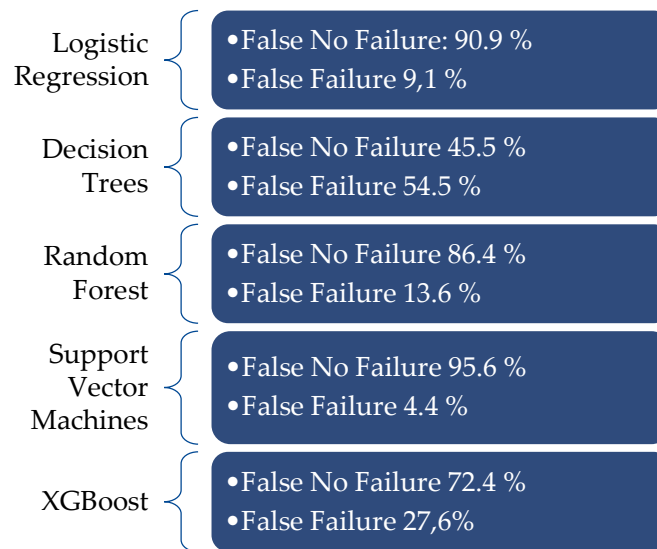


Figure 20. The relationship of False No Failures to False Failures.

6.4 Hyperparameter Optimization

The results in this section are the three best performing algorithms from the first test after they have gone through hyperparameter optimization, see figure 21.

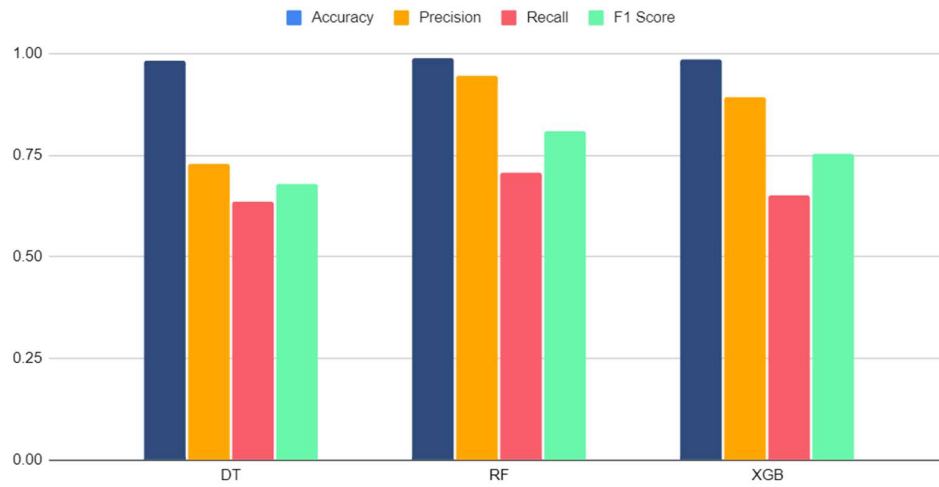


Figure 21. The results of the improved models.

Each algorithm had a small increase in accuracy, as seen in figure 22. Decision Trees had an improvement in precision, though with a lower recall. The F1 score increased slightly for each of the models.

Model	Accuracy base (%)	Accuracy hyperparameters (%)
DT	0.9776	0.981
RF	0.9889	0.9895
XGB	0.9855	0.9865

Figure 22. Slight increase in accuracy after hyperparameter tuning.

Iterative test runs with GridSearch showed that the following hyperparameter settings performed the best, though in some cases there were other options that performed equally well.

Decision Trees:

- `max_depth = 0`
- `max_features = None`
- `min_samples_leaf = 5`
- `min_samples_split = 13`

Random Forest

- `n_estimators = 200`
- `max_depth = none`
- `min_samples_split = 3`
- `min_samples_leaf = 1`
- `max_features = sqrt`

XGBoost

- `n_estimators = 200`
- `max_depth = 10`
- `learning_rate = 0.03`
- `subsample = 0.7`
- `colsample_bytree = 1.0`

The new training times and prediction times of the algorithms can be seen in figure 23. The training time for Decision Trees stayed roughly the same, and the prediction time was marginally quicker. For Random Forest the training time increased significantly from 0.6 seconds in the first test to 3.2 seconds during the second one with improved

hyperparameters. The prediction time also increased. XGBoost roughly doubled its training time, and the prediction time remained the same.

Model	Training Time (s)	Prediction Time (s)
DT	0.017282	0.00049
RF	1.518638	0.024902
XGB	0.220328	0.002796

Figure 23. The training and prediction times in seconds.

Figure 24 shows the memory use of the models after changing the hyperparameters. The increase in memory usage for training the model was very noticeable for the Random Forest algorithm with the number of trees in the forest being set to 200. XGBoost also had an increase. Decision Trees still used a very small amount of memory after the hyperparameter changes.

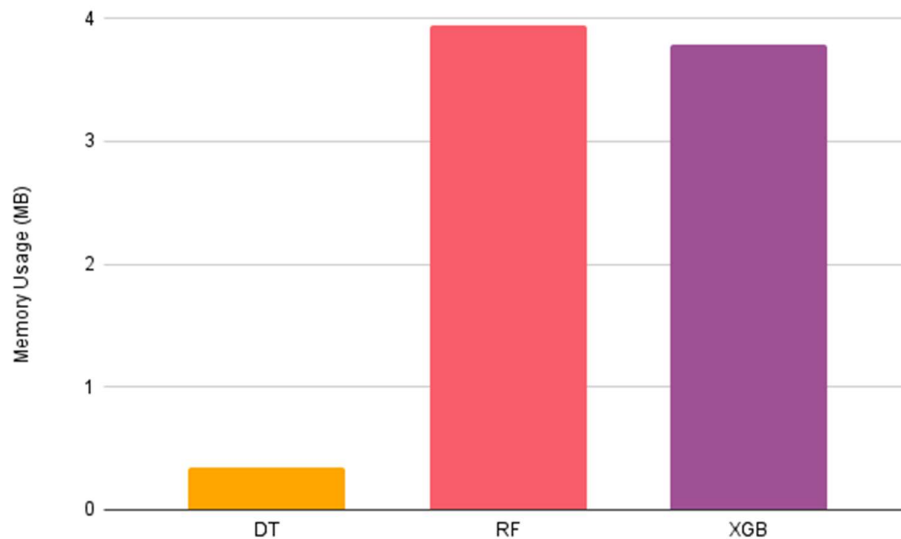


Figure 24. Memory use of each algorithm during training.

7 Discussion

This chapter discusses the results of the project, including the method and ethical considerations.

7.1 Analysis and discussion of results

This section of the paper discusses the results.

7.1.1 Base Algorithm Testing

The best performing algorithm during the initial tests, when it comes to predicting equipment failures, was Random Forest closely followed by XGBoost. Accuracy was high across all algorithms, but there was a big difference in precision, recall and F1 score. Recall was especially low for the non-tree-based models Logistic Regression and Support Vector Machines.

It was interesting to see the decision matrices for each model, since quite a few of them show a similar pattern. Except for Decision Trees, they all show a much higher failure when it comes to predicting “False No Failure” rather than “False Failure”. This means that they are more likely to miss machine failures rather than falsely claiming that there is one. Being more likely to miss machine failures is an undesired characteristic, since the missed maintenance could lead to increased downtime, higher repair costs, and potential safety risks for workers. It’s possible that the high number of failed “False No Failure” predictions might be due to the dataset being very imbalanced. Only 339 out of 10,000 data points are labeled as machine failures, so it’s possible that there might be too few samples of a certain type of machine failure for the models to properly learn to identify it.

Looking at the training and prediction times for the algorithms there are some that perform a bit better than others. Overall, the training and prediction times are low, with XGBoost, Logistic Regression and Decision Trees being especially quick at training. Support Vector Machines stood out at the slowest both when it comes to training and prediction.

When it comes to memory use Random Forest and XGBoost took more memory for training the model. This is expected due to them creating a collection of trees rather than just one.

7.1.2 Hyperparameter Optimization

XGBoost and Random Forest were selected for further testing due to having the best results in the base evaluation. Decision Trees was also selected due to having good performance metrics, a very low training and prediction time as well as low memory usage during training.

Using GridSearch for iterating through the hyperparameters to find a good setup was very useful. It saved a lot of time compared to testing each parameter by itself. All three models showed improvement during the optimization, though the improvements were rather small. One of the insights I gained from the optimization was how the increased number of parameters affect the training time, prediction time and memory use during training for the algorithms. It became very clear that Random Forest and XGBoost had different qualities, even though their metrics looked similar. Random Forest took a lot longer to train compared to XGBoost, and the prediction time is longer as well.

The amount of memory required during training was similar for Random Forest and XGBoost and remained very low for Decision Trees. This is to be expected since Random Forest and XGBoost are a collection of many trees, so they will take up more memory compared to a single Decision tree. Decision Trees also had the shortest training and prediction times.

There was no significant change to the confusion matrices after the optimization. They still held a large number of “False No Failures” compared to the “False Failures”.

7.2 Project method discussion

This section will contain an analysis and discussion of the project's method, approach, and metrics. In chapter 3.2 the project was divided into two sections, an initial literature study to better understand the topic and its challenges and a practical evaluation of the algorithms.

One of the biggest challenges of this project occurred right at the start. I was fortunate enough to get in contact with Knowit who suggested an exam project targeting AI and predictive maintenance. The details of the project were not decided, so my first task was to identify an interesting problem to work with. After some initial study into predictive maintenance, I came across a seminar discussing the topic, and observed that there were a lot of questions from what seemed to be smaller

companies. This led me to consider smaller companies I have worked for myself, and how they might have to approach the topic of machine learning in the years to come. Further reading revealed some interesting challenges to investigate. Though one of the main challenges for SMEs, collecting data, was considered too time consuming for the practical aspect of this project.

The gathering of information during the initial literature study was demanding but also very interesting. A significant number of hours have been spent on this phase. However, it was necessary to gain enough knowledge to move forward with the project.

Finding literature regarding predictive maintenance was rather straightforward, however finding sources more closely linked to small and medium sized enterprises proved to be more challenging. Quite a few of the papers pointed out the limited research on the subject, but also the importance of better understanding the challenges smaller companies might face. Finding appropriate sources became easier after reading a few papers and learning some of the foundational terminology. During the literature study I found important information about the challenges SMEs face, and how predictive maintenance in general works. I also researched machine learning in general and had good use of information about the different types there are when it came down to identifying what kind of algorithms that might work for this project.

The second phase of the project was to set up, train and evaluate the algorithms. This part of the project was very interesting, giving new understanding in how to prepare and set up a dataset for training and evaluation. The overall method for evaluating the algorithms went rather well, however there were a few challenges along the way.

One of them being what dataset to use. It was decided early on that collecting and preparing data from scratch was unrealistic for a project of this length. After a discussion with the supervisor at the company a number of possible datasets were selected. I went through them and set aside a few possible choices for further evaluation. Out of them the AI4I dataset was a good candidate due to being a complete dataset targeted specifically towards predictive maintenance. The dataset being synthetical is interesting due to SMEs often having challenges when it comes to having useful data at hand.

There are many types of algorithms, so they had to be cut down to a manageable number for this project. After some research I could pinpoint what type of machine learning problem I was dealing with, and from there work out a selection of suitable algorithms to work with. Overall, the practical tests of the algorithms went very well.

7.3 Scientific discussion

This chapter analyzes the project results and also provides a discussion about synthetic datasets.

7.3.1 Analyzing the Results

Looking back at the challenges common for many SMEs that were identified during the literature study, there are several qualities that the ideal algorithm should have. Being effective when it comes to predicting when a machine failure has occurred is crucial. Random Forest got marginally better overall metrics compared to XGBoost, partly due to a higher F1 score. XGBoost on the other hand got the highest ROC-AUC score, indicating that it's great at separating whether a machine failure has occurred or not. An easier setup might be of interest for smaller companies if they have limited expertise but want to set up an ML model. Decision Trees and Random Forests felt easier and more intuitive for a beginner to adjust with hyperparameters compared to XGBoost.

During the literature study it was clear that SMEs are more likely to be limited by resources compared to larger companies. Decision Trees performed well when considering training and prediction times as well as memory use during training.

As a conclusion, I would recommend XGBoost despite having marginally less optimal metrics compared to Random Forest. The lower training time for XGBoost compared to Random Forest becomes rather noticeable when the complexity of the model increased. However, if resources are very limited Decision Trees performed best out of the simpler models.

7.3.2 Synthetic Datasets

I found it interesting to read about synthetic datasets when selecting the dataset for the project. Though a deeper dive into the topic was set aside, it still had some interesting points from the perspective of smaller companies. One challenge for SMEs is the limited access to usable data,

and it's also difficult for many reasons to share real company data with other companies even though cooperation between smaller companies might be a good road forward. However, using real data to train a ML model that is then in turn used to create a synthetic dataset that can be shared might be an interesting option. This approach could benefit many small companies and allow sharing of useful datasets without releasing sensitive company information.

7.4 Consequence analysis

There are several smaller aspects of a predictive maintenance system that then get integrated into a bigger solution. There is the gathering of data with IoT technology, preprocessing the data so it can be used, building and testing the model, and finally integrating the model into the maintenance system. This study has been focused on preprocessing, building models, and evaluating them. Even though this has been a big task in itself, it is just a part of setting up an entire predictive maintenance system. However, having a good model for the predictions is a crucial step, since without effective predictions the maintenance system would be of little use to a company.

7.5 Ethical and societal discussion

Predictive maintenance has the potential to bring both positive and negative aspects to society. There are many benefits when able to properly predict the ideal time for maintenance. With less downtime and fewer interruptions, the productivity is likely to improve. This can also lead to better quality of services, making sure that they are more reliable for customers. There could also be environmental benefits when the number of unnecessary repairs and replacements are kept low. Although the use of resources and energy to run the predictive maintenance system would also have to be considered when evaluating if there is an environmental benefit. While the goal of preventive maintenance is to avoid emergency breakdowns, it also tends to replace components or run maintenance too early.

On the other hand, the reduction in maintenance could impact jobs, with the risk of maintenance staff losing their jobs. This is a concern for many who fear that automation and AI might impact certain parts in a negative way. One solution might be to retrain and allow the employees to perform other tasks instead. It's also worth noting that predictive

maintenance is not a strict replacement of reactive and preventive maintenance, but rather a complement. Unplanned breakdowns are still likely to occur, and in some businesses preventive maintenance might still be the better option. For example, when it comes to the maintenance of a commercial burner it would be impractical to close the whole system down to replace single components at the ideal time. Each shutdown is very costly, so maintenance is usually planned to a few concentrated maintenance sessions where everything is looked over. It is also possible that new jobs will be created in the area of predictive maintenance, however they might be jobs that require a workforce with specific skills.

Another positive aspect of predictive maintenance is that it might lower the risk of accidents in the workspace due to continuous monitoring of the system. This could improve safety.

8 Conclusions

The first aim of this project was to gather information regarding predictive maintenance and identify challenges for SMEs. This was successful as several potential problems for SMEs were found in the literature.

The second goal was to choose and test machine learning algorithms to investigate how useful they are. Five machine learning algorithms were successfully trained and tested, with three of them being further studied by adjusting their hyperparameters. The goal of finding the best algorithm when it comes to accuracy and effectiveness was achieved with XGBoost performing slightly better than Random Forest.

Lastly, for the third goal the algorithms were compared in terms of memory use, training time and prediction time. This was also successful and offered some extra value to Decision Trees with its low training times, prediction times and memory use.

8.1 Future work

It's possible that more data points containing machine failures would improve the learning of the ML models. Collecting and preparing more datapoints for the dataset could be a good area of future work.

The models evaluated in the project could be further tested by using real data to confirm that the algorithms that worked well on the synthetic dataset also predict well in that situation. During this project all algorithms have been run with the same features to make the tests as similar as possible. However, it's possible they could be further improved by tweaking the models, perhaps testing if a different feature selection might give better results for certain algorithms.

One clear area for further work is access to suitable datasets for SMEs. Several of the papers during the literature study listed this as a challenge. More research specifically targeted towards SMEs or improved ways to gather data locally might be of interest. Creating and sharing more public datasets would also be an area to work on, including synthetic datasets in situations where sharing company data is difficult.

References

- [1] L. De Propriis and D. Bailey, *Industry 4.0 and Regional Transformations*, Abingdon, UK: Routledge, Taylor & Francis Group, 2020. Available online: <https://directory.doabooks.org/handle/20.500.12854/27435>. Accessed 2024-05-13.
- [2] J. Passlick, S. Dreyer, D. Olivotti, L. Grützner, D. Eilers, and M. H. Breitner, 'Predictive maintenance as an internet of things enabled business model: A taxonomy' *Electronic Markets: The International Journal on Networked Business*, vol. 31, no. 1, pp. 67-87, 2021. doi: 10.1007/s12525-020-00440-5.
- [3] IoT Analytics, 'Predictive maintenance market, 2022 URL: <https://iot-analytics.com/predictive-maintenance-market/> Accessed 2024-05-13.
- [4] Next Move Strategy Consulting, 'Predictive maintenance market' URL: <https://www.nextmsc.com/report/predictive-maintenance-market> Accessed 2024-05-13.
- [5] European Commission, 'SME definition' URL: https://single-market-economy.ec.europa.eu/smes/sme-definition_en Accessed 2024-05-13.
- [6] EUR-Lex, 'Commission Recommendation of 6 May 2003 concerning the definition of micro, small and medium-sized enterprises,' *Official Journal of the European Union*, 2003. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32003H0361> Accessed 2024-05-13.
- [7] Thyago P. Carvalho, Fabrizzio A. A. M. N. Soares, Roberto Vita, Roberto da P. Francisco, João P. Basto, Symone G. S. Alcalá, 'A systematic literature review of machine learning methods applied to predictive maintenance' *Computers & Industrial Engineering*, vol. 137, November 2019, Elsevier Ltd. DOI: 10.1016/j.cie.2019.106024.
- [8] MDPI, 'Predictive Maintenance in Industry 4.0 for the SMEs: A Decision Support System Case Study Using Open-Source

Software' 2021 URL: <https://www.mdpi.com/2411-9660/7/4/98#:~:text=Predictive%20maintenance%20has%20many%20benefits,improve%20customer%20service%20%5B29%5D>
Accessed 2024-05-13.

- [9] IBM, 'What is reactive maintenance?' URL: <https://www.ibm.com/topics/reactive-maintenance> Accessed 2024-05-13.
- [10] Martin Pech, Jaroslav Vrchota, Jiří Bednář, 'Predictive Maintenance and Intelligent Sensors in Smart Factory: Review' Sensors, vol. 21, iss. 4, p. 1470, 2021, MDPI AG. DOI: 10.3390/s21041470
- [11] Aysegul Ucar, Mehmet Karakose, Necim Kırımça, 'Artificial Intelligence for Predictive Maintenance Applications: Key Components, Trustworthiness, and Future Trends' Applied Sciences, vol. 14, iss. 2, p. 898, 2024, MDPI AG. DOI: 10.3390/app14020898
- [12] Berkley School of Information, 'What Is Machine Learning (ML)? ' URL: <https://ischoolonline.berkeley.edu/blog/what-is-machine-learning/> Accessed 2024-05-14.
- [13] Medium, 'Supervised and Unsupervised Learning (an Intuitive Approach)' URL: <https://medium.com/@metehankozan/supervised-and-unsupervised-learning-an-intuitive-approach-cd8f8f64b644>
Accessed 2024-05-13.
- [14] Seldon, 'Supervised vs Unsupervised Learning Explained' URL: <https://www.seldon.io/supervised-vs-unsupervised-learning-explained> Accessed 2024-05-13.
- [15] Datacamp, 'Classification in Machine Learning: An Introduction' URL: <https://www.datacamp.com/blog/classification-machine-learning> Accessed 2024-05-14.
- [16] BuiltIn, 'When and Why to Standardize Your Data' URL: <https://builtin.com/data-science/when-and-why-standardize-your-data> Accessed 2024-05-14.

- [17] Real Python, 'Split Your Dataset With scikit-learn's `train_test_split()`' URL: <https://realpython.com/train-test-split-python-data/> Accessed 2024-05-14.
- [18] SCIKIT Learn, '1.1 Linear Models' URL: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression Accessed 2024-05-14.
- [19] IBM, 'What is logistic regression?' URL: <https://www.ibm.com/topics/logistic-regression> Accessed 2024-05-14.
- [20] Spiceworks, 'What Is Logistic Regression? Equation, Assumptions, Types, and Best Practices' URL: <https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-logistic-regression/> Accessed 2024-05-14.
- [21] IBM, 'What is a decision tree?' URL: https://www.ibm.com/topics/decision-trees?mhsrc=ibmsearch_a&mhq=decision%20trees Accessed 2024-05-14.
- [22] IBM, 'What is random forest?' URL: https://www.ibm.com/topics/random-forest?mhsrc=ibmsearch_a&mhq=random%20forest Accessed 2024-05-14.
- [23] IBM, 'What are support vector machines (SVMs)?' URL: https://www.ibm.com/topics/support-vector-machine?mhsrc=ibmsearch_a&mhq=Support%20Vector%20Machines Accessed 2024-05-14.
- [24] Medium, 'XGBoost: A Deep Dive into Boosting' URL: <https://medium.com/sfu-csmp/xgboost-a-deep-dive-into-boosting-f06c9c41349> Accessed 2024-05-14.
- [25] Neptune, 'How to Compare Machine Learning Models and Algorithms' URL: <https://neptune.ai/blog/how-to-compare-machine-learning-models-and-algorithms> Accessed 2024-05-14.

- [26] Medium, 'Accuracy, Precision, Recall, F-1 Score, Confusion Matrix, and AUC-ROC' URL: <https://medium.com/@riteshgupta.ai/accuracy-precision-recall-f-1-score-confusion-matrix-and-auc-roc-1471e9269b7d> Accessed 2024-05-14.
- [27] Neptune, 'Performance Metrics in Machine Learning [Complete Guide]' URL: <https://neptune.ai/blog/performance-metrics-in-machine-learning-complete-guide> Accessed 2024-05-14.
- [28] Neptune, 'F1 Score vs ROC AUC vs Accuracy vs PR AUC: Which Evaluation Metric Should You Choose?' URL: <https://neptune.ai/blog/f1-score-accuracy-roc-auc-pr-auc> Accessed 2024-05-14.
- [29] Kwaku Adu-Amankwa, Ashraf K.A. Attia, Mukund Nilakantan Janardhanan, Imran Patel, 'A predictive maintenance cost model for CNC SMEs in the era of industry 4.0' *The International Journal of Advanced Manufacturing Technology*, vol. 104, no. 9-12, pp. 3567-3587, 2019, Springer London. DOI: 10.1007/s00170-019-04094-2.
- [30] Kwaku Adu-Amankwa, Ashraf K.A. Attia, Mukund Nilakantan Janardhanan, Imran Patel, 'A predictive maintenance cost model for CNC SMEs in the era of industry 4.0' *The International Journal of Advanced Manufacturing Technology*, vol. 104, no. 9-12, pp. 3567-3587, 2019, Springer London. DOI: 10.1007/s00170-019-04094-2.
- [31] Vrinda Rastogi, Sahima Srivastava, Manasi Mishra, Rachit Thukral, 'Predictive Maintenance for SME in Industry 4.0' 2020 Global Smart Industry Conference (GloSIC), pp. 382-390, Nov. 2020, IEEE. DOI: 10.1109/GloSIC50886.2020.9267844.
- [32] M. Khan, A. Ahmad, F. Sobieczky, M. Pichler, B.A. Moser, I. Bukovsky, 'A Systematic Mapping Study of Predictive Maintenance in SMEs' *IEEE Access*, vol. 10, pp. 88738-88749, 2022, IEEE. DOI: 10.1109/ACCESS.2022.3200694.
- [33] Stephan Matzka, 'Explainable Artificial Intelligence for Predictive Maintenance Applications' in 2020 Third International Conference

on Artificial Intelligence for Industries (AI4I), pp. 69-74, Sep. 2020, IEEE. DOI: 10.1109/AI4I49448.2020.00023.

- [34] Nist, 'Research Methods' URL:
<https://www.nist.gov/itl/iad/visualization-and-usability-group/research-methods> Accessed 2024-05-14.
- [35] UC Irvine, 'AI4I 2020 Predictive Maintenance Dataset' URL:
<https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset> Accessed 2024-05-17.
- [36] Medium, 'Top 6 Machine Learning Algorithms for Classification How to Build a Machine Learning Model Pipeline in Python' URL:
<https://towardsdatascience.com/top-machine-learning-algorithms-for-classification-2197870ff501> Accessed 2024-05-17.
- [37] Medium, 'How to Choose the Best Algorithm for Your Machine Learning Project Optimizing ML Model Performance: A Guide to Algorithm Selection' URL:
https://medium.com/@BH_Chinmay/how-to-choose-the-best-algorithm-for-your-machine-learning-project-abd271ce6b11
Accessed 2024-05-17

Appendix A: ROC-AUC Curves

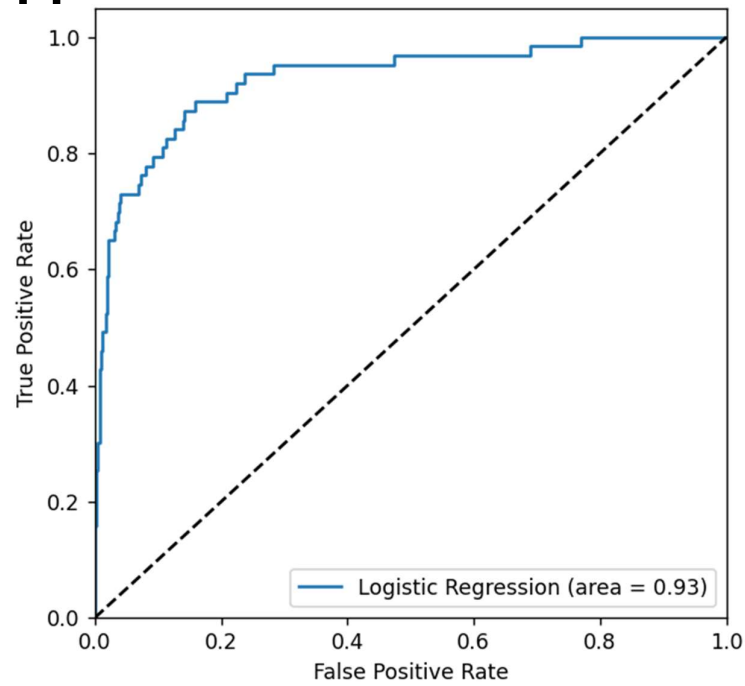


Figure 25. Logistic Regression

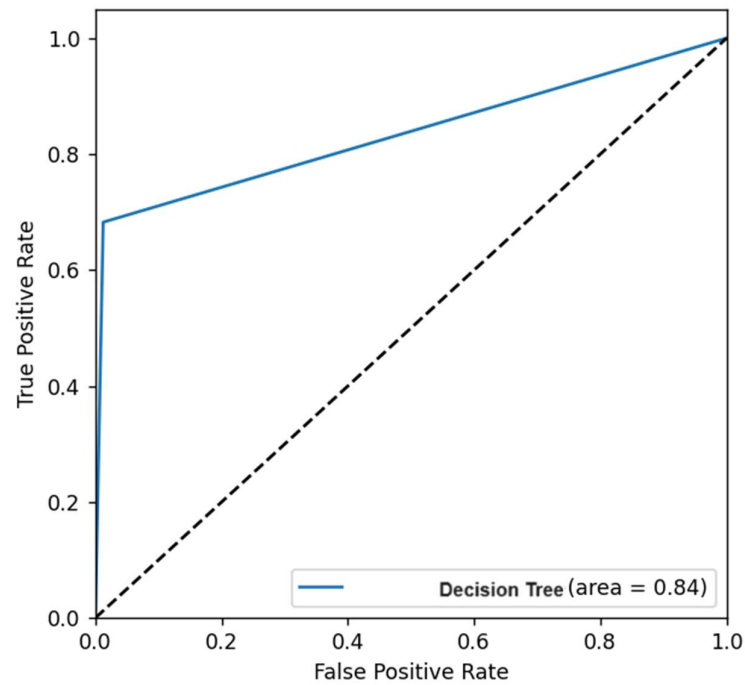


Figure 26. Decision Trees

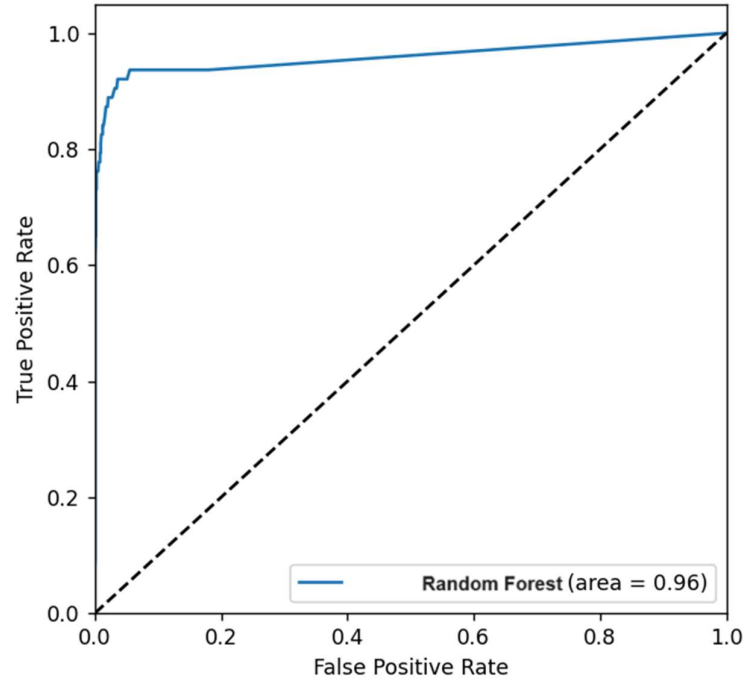


Figure 27. Random Forest

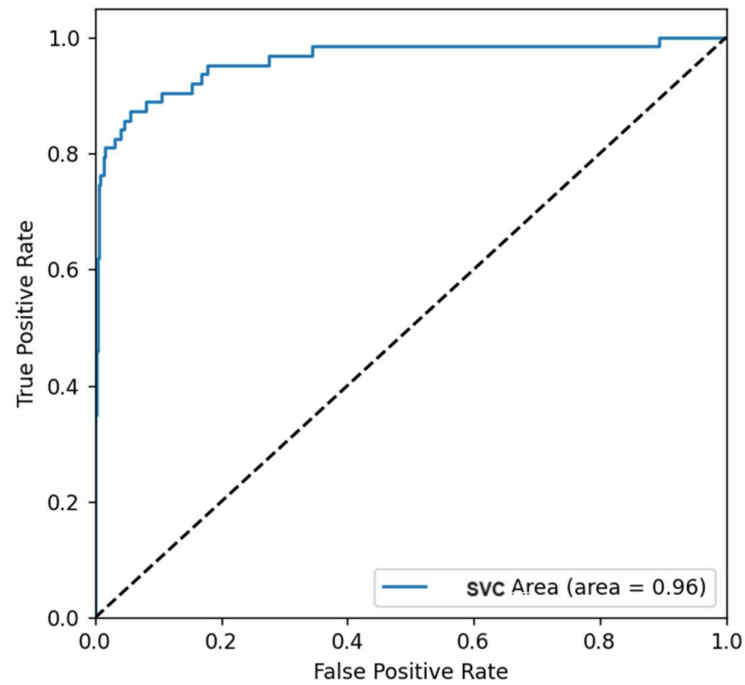


Figure 28. Support Vector Machines

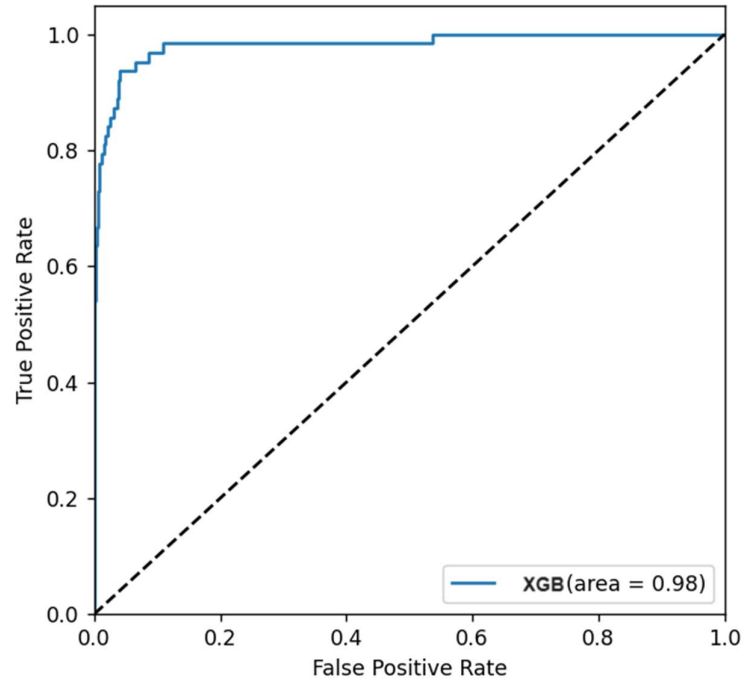


Figure 29. XGBoost

Appendix B: Source Code

<https://github.com/gazeup/exjobb>